ORQLAND Real-Time Chat Platform

Complete Technical Specification, Architecture Blueprint, Configuration & Operational Guide

ARCHITECTURE	FRONTEND FRAMEWORK	BACKEND SERVICE	DATA PERSISTENCE
Monorepo (PNPM)	**Next.js 16 (App Router)**	**NestJS 11 + Socket.IO**	**Postgres 16 + Redis 7**

1. Executive Summary & Core Objectives

ORQLAND is an enterprise real-time messaging platform enabling direct communication between business accounts and verified clients. The system delivers sub-50ms messaging latency, bi-directional live presence tracking, 48-hour automated media lifecycle management, and a robust offline-resilient development mode.

Sub-50ms WebSocket Mesh

WebSocket connections backed by Socket.IO with Redis adapter for distributed room fan-out and zero-drop message publishing.

Dual-Stack Resilience

Operates in fully provisioned PostgreSQL 16 + Redis production mode, or seamless auto-fallback in-memory mode for offline development.

Security & Policy Guardrails

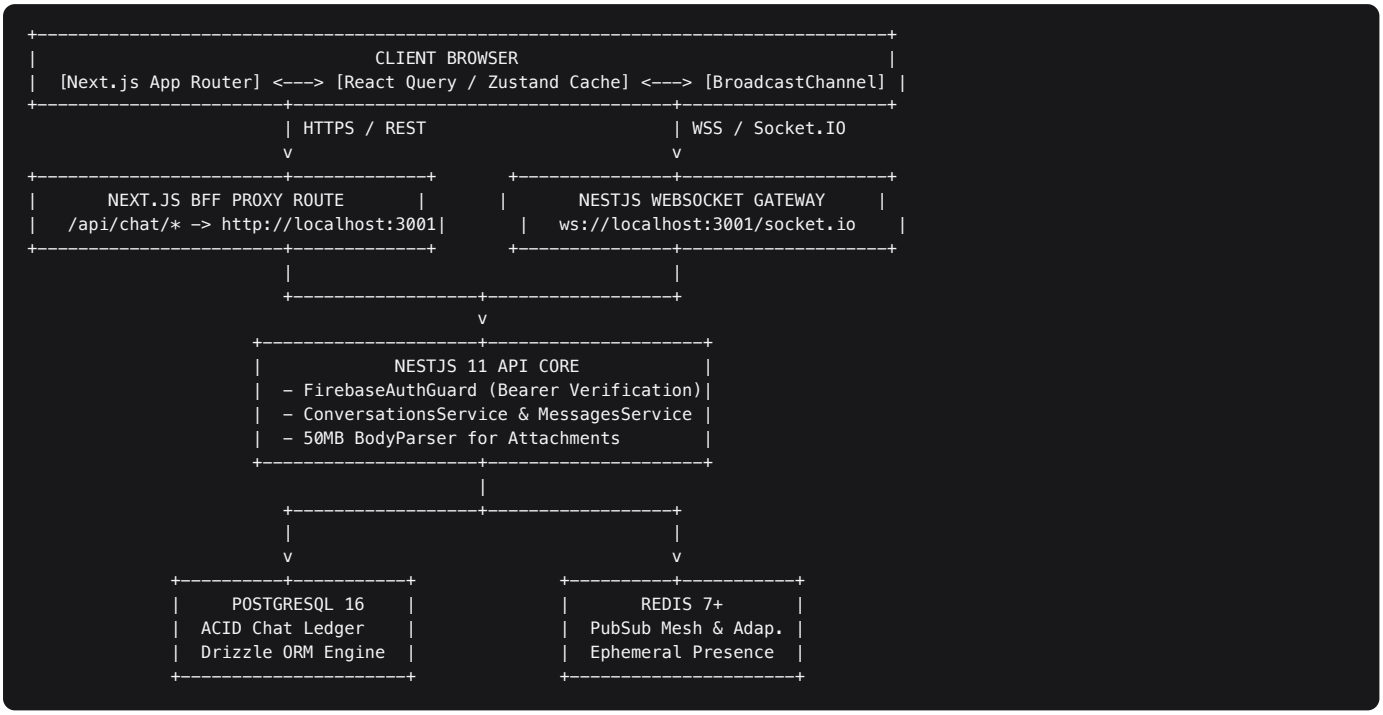
Real-time sensitive data interception (block OTPs, card numbers, passwords) combined with 48-hour automated media purging.

2. Full Technology Stack

Tier / Component	Technology & Version	Responsibility & Architectural Fit
Frontend Layer	Next.js 16.4.3, React 19.3, TailwindCSS v4	Server-rendered React components, optimistic caching, Discord-style in-chat media composer, responsive mobile drawer view.
State & Client Cache	TanStack Query v5, Zustand v5, BroadcastChannel	Zero-latency optimistic UI updates, cross-tab synchronization without redundant network refetches, active window notification suppression.
Backend Service	NestJS 11, Express 5, Node.js 22 LTS	Modular micro-monolith: dependency injection, validation pipes, global exception filters, REST controllers, and WebSocket gateway.
Real-Time Layer	Socket.IO v4, @socket.io/redis-adapter	Multiplexed full-duplex messaging, room partitioning (`conversation:{id}`), ephemeral heartbeat presence, and typing indicators.
Database & ORM	PostgreSQL 16, Drizzle ORM v0.38, Drizzle Kit	Type-safe schema definitions, ACID transactions for read cursors, monotonic message sequences, and JSON aggregation.
Cache & Distributed PubSub	Redis 7+, ioredis v5.4	Socket.IO cluster state, horizontal gateway scaling, active session tracking, and idempotency key locks.
Identity & Auth	Firebase Authentication, Firebase Admin v13	Client-side Google/Email auth token generation, server-side cryptographically verified JWT bearer tokens with internal UUID mapping.
Shared Contracts	`@orqland/contracts` (TypeScript)	Shared DTOs, interfaces, and validation contracts published across frontend and backend workspaces.

3. High-Level System Architecture

The system follows a BFF (Backend-For-Frontend) proxy pattern where Next.js proxies authenticated requests to NestJS, eliminating cross-origin browser credential hurdles while preserving end-to-end type safety.



4. End-to-End Workflow Lifecycles

A. Message Dispatch & Delivery Lifecycle

Every message undergoes an optimistic UI sequence with idempotency protection to prevent duplicate sends across flaky mobile networks:

1. **Optimistic Stage:** Client generates temporary UUID (`temp-timestamp-rand`) with status `'sending'` and renders immediately in the chat thread.
2. **REST Ingestion:** POST to `/api/chat/send` with `idempotencyKey`. NestJS verifies identity and commits transaction.
3. **Receipt Confirmation:** Server returns confirmed `ChatMessage` entity; client updates status to `'sent'`.
4. **WebSocket Fan-Out:** NestJS broadcasts `message:created` to the Socket.IO room `conversation:{id}`.
5. **Delivery & Read Receipts:** Recipient client emits `message:delivered`. Upon window focus, `/api/chat/read` marks messages as `'read'` and zeroes unread counters.

B. Single-Session Conversation Rule

To maintain a clean communication history, clients are strictly restricted to **one active dialogue session** per business partner. When a client initiates contact via `/api/chat/initiate`:

- If an active conversation already exists, the server returns the existing session and updates the workspace category rather than duplicating records.
- If a previous session was closed (`status = 'ended'`), the user is prompted with a modal to reopen the conversation with the updated topic.

C. 48-Hour Media Lifecycle (Auto-Deletion)

Uploaded attachments (images and PDF documents) are subject to a strict 48-hour retention window. The database assigns `expiresAt = uploadedAt + 48h`. Scheduled cron workers evaluate `expiresAt <= NOW()`, purge binary assets from object storage, and flag `is_expired = true`.

5. Configuration & Environment Variables

All configurable parameters are centralized in `.env` files. A comprehensive configuration blueprint is provided below:

```
# --- Network & Ports ---
PORT=3001
NODE_ENV=production
FRONTEND_URL=https://chat.orqland.com
NEST_API_URL=http://backend:3001/api/v1
NEXT_PUBLIC_SOCKET_URL=https://chat.orqland.com

# --- Relational Database: PostgreSQL 16 ---
DATABASE_URL=postgres://orqland_user:SecretSecurePass123@postgres-host:5432/orqland_prod
POSTGRES_DB=orqland_prod
POSTGRES_USER=orqland_user
POSTGRES_PASSWORD=SecretSecurePass123

# --- Distributed Cache & PubSub: Redis 7 ---
REDIS_URL=redis://:RedisSecurePass@redis-host:6379/0

# --- Object Storage (AWS S3 / Cloudflare R2 / GCS) ---
STORAGE_BUCKET=orqland-chat-attachments-prod
STORAGE_REGION=us-east-1
STORAGE_ENDPOINT=https://<account-id>.r2.cloudflarestorage.com
STORAGE_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
STORAGE_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY

# --- Firebase Admin Service Account (JWT Verification) ---
FIREBASE_PROJECT_ID=openpurpl-f8dd3
FIREBASE_CLIENT_EMAIL=firebase-adminsdk-xxx@openpurpl-f8dd3.iam.gserviceaccount.com
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\nMIIEvgIBADANBgkqhkiG9w0BAQEFAASCBKgwggSkAgEAAoIBAQC...\n-----END PRIVATE KEY-----\n"

# --- Client-Side Firebase Public Credentials ---
NEXT_PUBLIC_FIREBASE_API_KEY=AIzaSyA_ExamplePublicWebKey
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=openpurpl-f8dd3.firebaseapp.com
NEXT_PUBLIC_FIREBASE_PROJECT_ID=openpurpl-f8dd3
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=openpurpl-f8dd3.firebasestorage.app
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=1029384756
NEXT_PUBLIC_FIREBASE_APP_ID=1:1029384756:web:abcdef123456

# --- Business Operator Account ---
BUSINESS_EMAIL=mailtosidharth.me@gmail.com
```

6. API Reference (BFF & REST Endpoints)

Method	Endpoint	Auth Required	Action & Response
GET	`/api/chat/list?email=:email`	Bearer Token / Session	Returns active dialogue list sorted by `lastMessageTimestamp`.
POST	`/api/chat/initiate`	Bearer Token / Session	Opens or retrieves unique conversation for client-business pair.
POST	`/api/chat/send`	Bearer Token / Session	Dispatches message with attachments (up to 50MB payload).
POST	`/api/chat/read`	Bearer Token / Session	Marks conversation unread count as 0 and flags messages `'read'`.
POST	`/api/chat/reopen`	Bearer Token / Session	Restores an `ended` conversation session to `active` state.
POST	`/api/chat/end`	Bearer Token / Session	Gracefully closes conversation session; disallows further messages.
POST	`/api/chat/edit-message`	Sender Only (≤ 15 min)	Updates content of message within permitted 15-minute grace window.
GET	`/api/health`	Public	Returns system status, memory allocation, CPU load, and DB connectivity.

7. Security Architecture & Threat Mitigation

Payload Content Inspection

Client and server regex monitors evaluate messages for sensitive credentials (credit card PANs, CVVs, passwords, PINs, and OTPs). Prohibited messages are blocked before database insertion.

Dual-Layer Authentication Guard

Incoming requests are validated via `FirebaseAuthGuard` using Firebase Admin cryptographic verification, mapping external UIDs to internal non-enumerable UUID identifiers.

Idempotency Protection

Unique composite index `(conversation_id, idempotency_key)` ensures duplicate HTTP dispatches from retry loops do not yield duplicated messages.

Presigned Direct Uploads

Heavy multimedia binary streams bypass application web servers; temporary scoped presigned URLs write directly to object storage with 48h TTL expiration.

8. Production Deployment & Operational Runbook

A. Building the Monorepo

```
# 1. Install production dependencies with strict lockfile
pnpm install --frozen-lockfile

# 2. Build shared contracts, NestJS backend, and Next.js frontend
pnpm build

# 3. Execute backend automated test suite
pnpm test
```

B. Database Migration & Initialization

```
# Generate and apply schema migrations to PostgreSQL
pnpm db:generate
pnpm db:migrate

# Seed default developer and business accounts
pnpm db:seed
```

C. Process Management via PM2 / Docker

For zero-downtime production deployments, launch backend and frontend instances using PM2 or Docker Compose:

```
# Start NestJS backend micro-monolith
pm2 start backend/dist/backend/src/main.js --name "orqland-api" -i max

# Start Next.js frontend SSR cluster
pm2 start frontend/node_modules/next/dist/bin/next --name "orqland-web" -- start frontend -p 3000
```

9. Verification & Health Monitoring

Inspect system metrics and active connection health via the built-in health probe endpoint:

```
curl -i http://localhost:3000/api/health

HTTP/1.1 200 OK
Content-Type: application/json

{
  "status": "healthy",
  "service": "orqland-chat-web",
  "uptime": 14528.2,
  "system": {
    "cpus": 8,
    "memory": { "usedMB": 72.4, "totalMB": 128.0 }
  }
}
```